

Aula 2 — Explorando e Operando o Sistema Linux

Navegação, manipulação de arquivos, edição de conteúdo, redirecionamento, pipes e gerenciamento de pacotes no Linux Mint

Introdução

Na Aula 1, vimos *onde* a nuvem existe fisicamente e *o que* ela oferece (IaaS, PaaS, SaaS). Mas há um detalhe que conecta tudo isso à prática: **quase toda infraestrutura de nuvem roda sobre Linux**, e é no terminal — não em interfaces gráficas — que analistas de ADS interagem com servidores, containers e pipelines de CI/CD.

Nesta aula, vamos abrir o Linux Mint por dentro: entender como seus arquivos são organizados, aprender a navegar, criar, mover e editar arquivos, e dar os primeiros passos com dois recursos que vão acompanhar vocês em toda a disciplina — **redirecionamento e pipes**. Fechamos com algo prático: como manter o sistema atualizado e instalar as ferramentas que vamos usar nas próximas aulas.

Parte 1 — A organização interna do Linux (FHS)

Quando vocês instalam um programa no Windows, ele geralmente cria uma pasta em C:\Program Files\. Mas onde ficam os *programas* no Linux? E onde ficam os *arquivos de configuração*? E os *logs* de erro de uma aplicação em produção? Existe uma lógica por trás disso, ou é cada distribuição fazendo do seu jeito?

Exemplo real

Imaginem um analista de DevOps investigando por que uma aplicação da **iFood** parou de responder em um servidor. Ele não vai abrir um "Explorador de Arquivos" clicando em pastas ao acaso — ele sabe, de cor, que os logs do sistema ficam em `/var/log`, que os arquivos de configuração do serviço ficam em `/etc`, e que os binários que ele pode executar estão em `/usr/bin` ou `/usr/local/bin`. Essa previsibilidade é o que permite que qualquer profissional, em qualquer servidor Linux do mundo — seja um servidor da AWS rodando a Mercado Livre, seja a VM de vocês no laboratório — saiba exatamente onde procurar.

Definição formal

Essa organização não é acidental: ela segue um padrão chamado **FHS (Filesystem Hierarchy Standard)**, mantido pela Linux Foundation. O FHS define *que tipo de arquivo* deve morar em *qual diretório*, garantindo que distribuições diferentes (Linux Mint, Ubuntu, Debian, o Linux que roda dentro de um container Docker) compartilhem uma estrutura previsível.

Os diretórios principais e sua função

Tudo no Linux começa em um único diretório: a **raiz**, representada por /. Todos os outros diretórios são "galhos" dessa raiz.

Diretório	Função	Exemplo prático
/	Raiz do sistema de arquivos — todos os outros diretórios partem daqui	—
/bin	Comandos essenciais que qualquer usuário pode executar (ls, cp, cat)	Hoje é um link simbólico para /usr/bin na maioria das distros modernas
/boot	Arquivos necessários para o Linux iniciar (kernel, bootloader)	vmlinux, grub
/dev	Representações de dispositivos de hardware como arquivos	/dev/sda (disco), /dev/null
/etc	Arquivos de configuração do sistema e dos serviços instalados	Configuração de rede, do Apache, do SSH
/home	Diretório pessoal de cada usuário	/home/nelson, /home/aluno
/lib	Bibliotecas compartilhadas usadas pelos programas do sistema	Equivalente às DLLs do Windows
/media	Ponto de montagem automático para pendrives, CDs, discos externos	Quando vocês conectam um pendrive, ele aparece aqui
/mnt	Ponto de montagem <i>manual</i> , usado temporariamente por administradores	Montar um disco extra para copiar arquivos
/opt	Software de terceiros instalado manualmente (fora do gerenciador de pacotes)	Muitas aplicações comerciais se instalam aqui
/proc	Sistema de arquivos "virtual" com informações do kernel e processos em execução	Não existe fisicamente no disco — é gerado pelo próprio sistema
/root	Diretório pessoal do superusuário (root) — diferente de /	Não confundir /root com /
/sbin	Comandos administrativos, geralmente restritos ao root	reboot, fdisk
/srv	Dados de serviços que o servidor oferece (sites, arquivos compartilhados)	Um servidor web pode servir arquivos daqui

Diretório	Função	Exemplo prático
/tmp	Arquivos temporários, apagados a cada reinicialização	Usado por programas para arquivos de curta duração
/usr	A maior parte dos programas e dados de uso comum do sistema	/usr/bin, /usr/lib, /usr/share
/var	Dados que variam com o tempo: logs, filas de e-mail, cache	/var/log — o primeiro lugar a olhar quando algo dá errado

Por que isso importa para DevOps: quando formos trabalhar com Docker, Terraform e AWS CLI, os arquivos de configuração (.tf, .yml, chaves de acesso) vão morar em convenções parecidas — dentro do seu /home, referenciando caminhos do sistema. Entender o FHS evita o erro clássico de iniciante: "salvei o arquivo, mas não sei onde foi parar".

Exercício 1 — Explorando o FHS

Na VM do laboratório, abram o terminal e executem, um de cada vez, anotando o que aparece:

```
ls /
```

```
ls /etc
```

```
ls /var/log
```

```
ls /home
```

Perguntas para pesquisar

1. Quantos itens existem dentro de /etc? Cite três nomes que vocês reconhecem ou conseguem supor a função.
2. Dentro de /var/log, existe algum arquivo com o nome syslog ou auth.log? O que vocês imaginam que ele guarda?
3. Existe uma pasta com o nome de vocês dentro de /home? O que isso sugere sobre como o Linux trata cada usuário?

Parte 2 — Navegação no terminal

Se não há ícones para clicar, como alguém "anda" pelas pastas de um sistema usando apenas texto?

Exemplo real

Times de DevOps administram, muitas vezes, dezenas de servidores **sem interface gráfica nenhuma** — apenas acesso remoto via terminal (SSH). Um engenheiro da Nubank conectado a um servidor de produção não tem "duplo clique": ele precisa saber, de memória, os comandos que o levam exatamente onde precisa ir, da forma mais rápida possível, porque cada segundo em um servidor de produção é sensível.

Definição formal e comandos

Comando	Função	Exemplo
<code>pwd</code>	<i>Print Working Directory</i> — mostra em qual pasta você está agora	<code>pwd</code>
<code>cd</code>	<i>Change Directory</i> — muda de pasta	<code>cd Documentos</code>
<code>cd ..</code>	Sobe um nível (vai para a pasta "pai")	<code>cd ..</code>
<code>cd ~</code> ou <code>cd (sozinho)</code>	Vai direto para o seu diretório pessoal (/home/seu_usuario)	<code>cd ~</code>
<code>cd -</code>	Volta para a última pasta em que você estava	<code>cd -</code>
<code>cd /</code>	Vai direto para a raiz do sistema	<code>cd /</code>
<code>ls</code>	Lista o conteúdo da pasta atual	<code>ls</code>
<code>ls -l</code>	Lista em formato detalhado (permissões, dono, tamanho, data)	<code>ls -l</code>
<code>ls -a</code>	Mostra também arquivos ocultos (que começam com .)	<code>ls -a</code>
<code>ls -la</code>	Combina os dois: lista detalhada incluindo ocultos	<code>ls -la</code>
<code>ls -lh</code>	Lista detalhada com tamanhos "legíveis" (KB, MB) em vez de bytes	<code>ls -lh</code>

Caminhos absolutos vs. relativos

Isso é um dos pontos que mais gera confusão no início — vale reforçar bem:

- **Caminho absoluto:** começa sempre a partir da raiz /. Funciona de qualquer lugar do sistema.
 - Exemplo: `cd /home/aluno/projetos`
- **Caminho relativo:** parte de onde você está agora. Muda de resultado dependendo da pasta atual.
 - Exemplo: se você já está em `/home/aluno`, basta `cd projetos`

Analogia: um caminho absoluto é como dar um endereço completo com CEP para um motorista de aplicativo que nunca esteve na cidade. Um caminho relativo é como dizer "vire a próxima esquina" — só funciona se ele já estiver no ponto de partida certo.

Exercício 2 — Navegando pelo sistema

1. Descubra em qual diretório você está agora (`pwd`).
 2. Vá até `/etc` usando um **caminho absoluto**.
 3. Volte ao seu diretório pessoal usando `cd ~`.
 4. Liste o conteúdo do seu diretório pessoal incluindo arquivos ocultos. Existe algum arquivo cujo nome começa com ponto (`.`)? Anote pelo menos dois.
 5. Use `cd` - duas vezes seguidas e observe o que acontece. Expliquem com suas palavras o comportamento desse comando.
-

Parte 3 — Manipulação de arquivos e diretórios

Como organizar, duplicar, mover ou apagar arquivos sem nunca clicar em um botão do mouse?

Exemplo real

Ao configurar um novo projeto de infraestrutura como código (o que veremos com Terraform mais adiante), o primeiro passo de qualquer profissional é sempre o mesmo: criar a estrutura de pastas do projeto, copiar arquivos de modelo (*templates*) e organizar tudo antes de escrever a primeira linha de código. Tudo isso é feito por linha de comando — em um servidor remoto, é a única opção disponível.

Comandos essenciais

Comando	Função	Exemplo
mkdir	Cria um novo diretório	mkdir projeto-nuvem
mkdir -p	Cria diretórios aninhados de uma vez, mesmo que os intermediários não existam	mkdir -p projeto/app/config
touch	Cria um arquivo vazio (ou atualiza a data de um existente)	touch anotacoes.txt
cp	Copia um arquivo	cp anotacoes.txt backup.txt
cp -r	Copia uma pasta inteira, com todo o conteúdo (<i>recursivo</i>)	cp -r projeto/ projeto-backup/
mv	Mover um arquivo/pasta — também usado para renomear	mv backup.txt arquivo-antigo.txt
rm	Remove um arquivo	rm arquivo-antigo.txt
rm -r	Remove uma pasta e todo o seu conteúdo	rm -r projeto-backup/
rmdir	Remove uma pasta, mas apenas se estiver vazia	rmdir pasta-vazia

Atenção redobrada com rm -r: diferente do "Excluir" do Windows, o Linux **não tem lixeira** por padrão no terminal. Um rm -r mal digitado pode apagar arquivos de forma permanente e irreversível. Times de produção evitam catástrofes assim com o hábito de sempre conferir com pwd e ls *antes* de rodar um comando destrutivo.

Exercício 3 — Organizando um projeto

1. Crie uma estrutura de pastas para um projeto fictício chamado loja-cloud, com subpastas src, config e logs, usando um único comando com `mkdir -p`.
 2. Dentro de src, crie três arquivos vazios: `app.py`, `README.md` e `.gitignore`.
 3. Copie o `README.md` para dentro da pasta config, renomeando a cópia para `README-config.md`.
 4. Mova o arquivo `.gitignore` para a pasta raiz do projeto (loja-cloud).
 5. Remova a pasta logs (que ainda está vazia) usando o comando mais apropriado — pense em qual comando é o mais seguro para esse caso.
-

Parte 4 — Visualizando e editando conteúdo

Como ler o conteúdo de um arquivo de configuração ou de um log de erro com centenas de linhas, sem precisar abrir um editor gráfico?

Exemplo real

Ao investigar uma falha em produção, é comum que o arquivo de log tenha milhares de linhas. Abrir esse arquivo inteiro de uma vez seria inviável — o profissional precisa de ferramentas que mostrem só um pedaço por vez, ou que permitam navegar rapidamente até o fim do arquivo (onde normalmente está o erro mais recente).

Comandos para visualizar

Comando	Função	Exemplo
cat	Mostra o conteúdo inteiro do arquivo de uma vez	cat app.py
less	Mostra o conteúdo "página por página", navegável (setas, q para sair)	less /var/log/syslog
head	Mostra as primeiras linhas do arquivo (10 por padrão)	head arquivo.txt
head -n 5	Mostra as 5 primeiras linhas	head -n 5 arquivo.txt
tail	Mostra as últimas linhas do arquivo (10 por padrão)	tail arquivo.txt
tail -n 20	Mostra as últimas 20 linhas	tail -n 20 arquivo.txt
tail -f	Acompanha o arquivo em tempo real , mostrando novas linhas assim que são escritas	tail -f /var/log/syslog

tail -f é o comando que qualquer analista deixa aberto em um terminal enquanto testa uma aplicação — é assim que se vê, ao vivo, o log de erro aparecendo no exato momento em que algo quebra.

Editando arquivos com nano

O Linux Mint já vem com o **nano**, um editor de texto simples que roda direto no terminal — sem mouse, sem janelas.

Comando/atalho	Função
nano arquivo.txt	Abre (ou cria) o arquivo para edição
Ctrl + O	Salva o arquivo (<i>Write Out</i>)
Ctrl + X	Sai do editor
Ctrl + K	Recorta a linha atual
Ctrl + U	Cola a linha recortada

Exercício 4 — Lendo e editando

1. Use nano para abrir o arquivo app.py (criado no exercício anterior) e escreva dentro dele a linha `print("Aplicação iniciada")`. Salve e saia.
 2. Confira o conteúdo com `cat app.py`.
 3. Use o terminal para gerar um arquivo com bastante conteúdo, rodando: `seq 1 200 > numeros.txt`
 4. Use `head -n 5 numeros.txt` e depois `tail -n 5 numeros.txt`. O que aparece em cada caso?
 5. Abra numeros.txt com less e naveguem com as setas do teclado. Saiam pressionando q.
-

Parte 5 — Redirecionamento e pipes

E se, em vez de só *ler* a saída de um comando na tela, eu quisesse *salvá-la* em um arquivo? Ou usar a saída de um comando como *entrada* de outro?

Exemplo real

Imaginem um pipeline de CI/CD (que veremos mais à frente) rodando testes automatizados a cada novo código enviado pela equipe. Ele não exibe o resultado apenas na tela — ele **redireciona** a saída para um arquivo de log, e depois **filtra** esse log em busca da palavra "ERROR" para decidir se o pipeline passa ou falha. Isso tudo acontece encadeando comandos simples, sem nenhuma interface gráfica.

Redirecionamento

Operador	Função	Exemplo
>	Redireciona a saída de um comando para um arquivo, substituindo o conteúdo existente	ls -la > lista.txt
>>	Redireciona a saída, adicionando ao final do arquivo (sem apagar o que já existe)	echo "nova linha" >> lista.txt
<	Usa o conteúdo de um arquivo como entrada de um comando	sort < numeros.txt

Pipes (|)

O pipe conecta a **saída** de um comando diretamente à **entrada** do próximo, sem precisar de um arquivo intermediário.

Comando útil com pipe	Função
grep	Filtra linhas que contêm um termo específico
wc -l	Conta quantas linhas foram recebidas
sort	Ordena as linhas recebidas
uniq	Remove linhas duplicadas consecutivas

Exemplos combinados:

Lista os arquivos da pasta atual e salva em um arquivo

```
ls -la > lista.txt
```

Busca, dentro do log do sistema, apenas as linhas com a palavra "error"

```
cat /var/log/syslog | grep "error"
```

Conta quantos processos estão rodando no sistema

```
ps aux | wc -l
```

Lista os arquivos, filtra os que têm "log" no nome, e ordena

```
ls -la | grep "log" | sort
```

Como ler um comando com pipe: leiam da esquerda para a direita como uma linha de produção. Cada comando recebe o que o anterior "produziu" e entrega um resultado transformado para o próximo.

Exercício 5 — Filtrando e combinando

1. Gere uma lista de 100 números com `seq 1 100 > numeros.txt`.
 2. Use `cat numeros.txt | grep "5"` — quais números aparecem? Por que o número 15 também aparece, mesmo a busca sendo por 5?
 3. Conte quantas linhas têm o dígito "5" usando pipe com `wc -l`.
 4. Use `ls -la /etc | grep ".conf"` e observem quantos arquivos de configuração aparecem.
 5. Crie um arquivo chamado `relatorio.txt` e, usando `>>`, adicione três linhas de texto diferentes em três comandos separados (`echo "linha 1" >> relatorio.txt` etc). Depois use `cat` para conferir se as três linhas foram mantidas na ordem certa.
-

Parte 6 — Atualizando o sistema e instalando aplicações

Quando vocês instalam um aplicativo no celular, ele vem de uma "loja" central, com atualizações automáticas. Existe algo parecido no Linux?

Exemplo real

Servidores em produção — incluindo os que rodam aplicações da iFood ou do Mercado Livre — raramente são atualizados "na mão", clicando em botões. Eles usam **gerenciadores de pacotes** via linha de comando, muitas vezes dentro de scripts automatizados, para garantir que toda a frota de servidores tenha exatamente as mesmas versões de software instaladas — isso é, inclusive, parte do que uma pipeline de DevOps automatiza.

Definição formal

O Linux Mint (assim como Ubuntu e Debian) usa o **APT** (*Advanced Package Tool*) como gerenciador de pacotes. Ele resolve automaticamente as dependências de cada programa e busca os arquivos em repositórios oficiais.

Comando	Função
<code>sudo apt update</code>	Atualiza a lista de pacotes disponíveis nos repositórios (não instala nada ainda)
<code>sudo apt upgrade</code>	Atualiza de fato os pacotes já instalados para suas versões mais recentes
<code>sudo apt install <pacote></code>	Instala um novo programa
<code>sudo apt remove <pacote></code>	Remove um programa (mantém arquivos de configuração)
<code>sudo apt purge <pacote></code>	Remove um programa e seus arquivos de configuração
<code>sudo apt autoremove</code>	Remove pacotes que não são mais necessários (dependências órfãs)
<code>apt search <termo></code>	Busca pacotes pelo nome ou descrição

Por que sudo? Instalar ou remover programas afeta o sistema inteiro, não só o seu usuário — por isso exige privilégios de administrador. `sudo` (*superuser do*) executa o comando seguinte "como root", pedindo sua senha antes.

Por que sempre update antes de upgrade? O update atualiza apenas a "lista de compras" disponível; o upgrade de fato instala as versões novas. Sem rodar o update primeiro, o sistema pode estar tentando instalar a partir de uma lista desatualizada.

Exercício 6 — Atualizando e instalando

1. Atualize a lista de pacotes do sistema: `sudo apt update`
 2. Instale o pacote `tree`, que permite visualizar a estrutura de pastas em formato de árvore:
`sudo apt install tree`
 3. Testem o programa recém-instalado dentro da pasta `loja-cloud` criada anteriormente:
`tree loja-cloud`
 4. Instale também o `htop`, um monitor de processos mais visual que o tradicional `top`:
`sudo apt install htop`
 5. Rodem `htop`, observem por 30 segundos os processos em execução e a memória usada, e saiam pressionando `q`.
 6. Busquem, sem instalar, se existe algum pacote relacionado a "git" disponível: `apt search git`
-

Prática integradora final

Combinando tudo o que vimos nesta aula, executem a sequência abaixo em ordem, anotando o resultado de cada etapa no roteiro de laboratório:

1. Vá para o diretório pessoal e crie a estrutura do projeto

```
cd ~
```

```
mkdir -p aula2/relatorios
```

2. Entre na pasta e gere um arquivo de dados

```
cd aula2
```

```
seq 1 50 > dados.txt
```

3. Filtre os números pares que contêm o dígito "2" e salve em outro arquivo

```
cat dados.txt | grep "2" > relatorios/numeros-com-2.txt
```

4. Conte quantas linhas esse novo arquivo tem

```
wc -l relatorios/numeros-com-2.txt
```

5. Visualize a estrutura final da pasta (usando o programa instalado no Exercício 6)

```
tree aula2
```

Pergunta final para reflexão em grupo: cada um dos comandos acima poderia ter sido feito clicando em uma interface gráfica. Na opinião de vocês, por que profissionais de nuvem e DevOps preferem fazer isso via terminal, mesmo sendo "mais difícil" no início? Pensem em escala (1 servidor vs. 1.000 servidores) para embasar a resposta.
